

Les props et les states

Table des matières

I. Contexte	3
II. Introduction aux Props et State	3
A. Présentation des Props	3
B. Présentation du State	4
C. Exercice : Quiz	6
III. Cycle de vie, vérification et communication bidirectionnelle	7
A. Cycle de vie et méthodes pour vérifier les props (PropTypes)	7
B. Communication bidirectionnelle des données (de parent à enfant / d'enfant à parent)	8
C. Exercice : Quiz	10
IV. L'essentiel	11
V. Auto-évaluation	11
A. Exercice	11
B. Test	12
Solutions des exercices	13

I. Contexte

Durée (en minutes) : 60

Environnement de travail : React Native 0.71.3

Pré requis : bases en React, être à l'aise avec les fonctions composantes.

Contexte

Dans le monde professionnel, créer des applications React Native maintenables et robustes est crucial pour assurer un développement efficace et une expérience utilisateur optimale. Dans ce contexte, maîtriser les concepts clés de React Native, tels que les props, le state et le cycle de vie des composants, est essentiel. En abordant ces notions à travers des composants fonctionnels et des hooks, nous vous donnerons des outils pour créer des applications performantes et faciles à gérer.

Nous explorerons le rôle des props et du state dans la gestion des données et la communication entre les composants. Nous verrons comment tirer parti des hooks pour gérer le cycle de vie et les états dans les composants fonctionnels. De plus, nous aborderons les techniques pour implémenter la communication bidirectionnelle des données, essentielle pour assurer une interaction fluide entre les composants parents et enfants.

En mettant l'accent sur les bonnes pratiques et les exemples pratiques, vous développerez une compréhension approfondie de ces concepts clés, vous permettant de créer des applications React Native de qualité professionnelle, tout en suscitant votre intérêt et en renforçant vos compétences en développement d'applications.

II. Introduction aux Props et State

A. Présentation des Props

Définition

Qu'est-ce qu'une prop ?

Les props sont des données passées d'un composant parent à un composant enfant dans React Native. Ils permettent de transmettre des informations et des fonctionnalités à travers les composants et sont utilisés pour rendre les composants réutilisables et modulaires. Les props sont immuables, ce qui signifie qu'un composant enfant ne peut pas modifier les valeurs des props reçues.

```
1 import React from 'react';
2
3 const Greeting = (props) => {
4   return <Text>Bonjour, {props.name}!</Text>;
5 };
6
7 export default Greeting;
```

Exemple

Utilisation des props dans un composant fonctionnel

Dans cet exemple, nous avons créé un composant fonctionnel Greeting qui accepte une propriété name et l'affiche dans un élément Text. Le prop name est transmis au composant enfant Greeting par le composant parent, comme il est possible de le voir dans le code suivant :

```
1 import React from 'react';
2 import { Text } from 'react-native';
3 import Greeting from './Greeting';
4
5 const App = () => {
6   return (
```

```

7 <View>
8 <Greeting name="Marie" />
9 <Greeting name="Pierre" />
10 </View>
11 );
12 };
13
14 export default App;

```

Conseil Utilisation de la décomposition pour les props

Pour améliorer la lisibilité et la propreté du code, il est possible d'utiliser la décomposition (destructuring) pour extraire les props directement dans les paramètres de la fonction du composant.

```

1 import React from 'react';
2 import { Text } from 'react-native';
3
4 const Greeting = ({ name }) => {
5   return <Text>Bonjour, {name}!</Text>;
6 };
7
8 export default Greeting;

```

B. Présentation du State

Définition Qu'est-ce que le state ?

Le state est un mécanisme permettant de stocker et de gérer les données internes d'un composant. Contrairement aux props, le state est mutable, ce qui signifie qu'il peut être modifié au cours du cycle de vie d'un composant. Le state est utilisé pour gérer les données qui peuvent changer au fil du temps et pour déclencher des mises à jour lorsque ces données sont modifiées.

Exemple Utilisation du state dans un composant fonctionnel avec useState

Dans cet exemple, nous allons créer un composant fonctionnel Counter qui utilise le hook useState pour gérer le state interne.

```

1 import React, { useState } from 'react';
2 import { View, Text, Button } from 'react-native';
3
4 const Counter = () => {
5   const [count, setCount] = useState(0);
6
7   return (
8     <View>
9       <Text>{count}</Text>
10      <Button
11        title="Incrémenter"
12        onPress={() => setCount(count + 1)}
13      />
14    </View>
15  );
16 };
17
18 export default Counter;

```

Fonctionnement du hook useState

Le hook `useState` est utilisé pour déclarer une variable d'état dans un composant fonctionnel. Il prend un argument, qui est la valeur initiale de l'état, et retourne un tableau contenant deux éléments : la valeur de l'état actuel et une fonction pour mettre à jour cet état. Dans l'exemple ci-dessus, nous avons utilisé `useState(0)` pour initialiser notre état `count` avec la valeur 0. Puis, nous avons utilisé la fonction `setCount` pour mettre à jour la valeur de `count` lorsque nous appuyons sur le bouton « Incrémenter ».

Gérer plusieurs états avec useState

Il est possible d'utiliser plusieurs appels à `useState` dans un même composant fonctionnel pour gérer différents états. Chaque appel à `useState` crée un nouvel état indépendant. Veillez à ne pas trop complexifier la gestion de l'état d'un seul composant. Si nécessaire, divisez votre composant en sous-composants pour simplifier la logique et améliorer la lisibilité du code.

Fondamental Le hook useEffect

Le hook `useEffect` est un outil puissant pour gérer les effets secondaires dans les composants fonctionnels de React Native. Les effets secondaires incluent les appels réseau, les abonnements et les mises à jour du DOM, qui sont généralement exécutés en réponse à des changements de props ou d'état. Le hook `useEffect` permet d'effectuer ces actions et de les synchroniser avec le cycle de vie du composant.

Exemple Utilisation du hook useEffect pour initialiser les données fetchées dans un composant fonctionnel

Dans cet exemple, nous allons utiliser le hook `useEffect` pour initialiser les données à afficher dans le composant. Lorsque le composant est monté pour la première fois, `useEffect` est appelé une fois avec un tableau de dépendances vide.

Un tableau de dépendances (ou « *dependency array* » en anglais) est un paramètre optionnel que l'on peut passer à la fonction `useEffect` dans un composant React. Ce tableau permet de spécifier les variables dont dépend l'effet déclenché par `useEffect`.

Le tableau de dépendances est utilisé pour optimiser les performances du composant, si l'une des variables spécifiées dans le tableau de dépendances change, alors la fonction `useEffect` sera appelée et l'effet associé sera déclenché à nouveau. Si aucune variable n'est spécifiée dans le tableau de dépendances, alors la fonction `useEffect` sera appelée à chaque fois que le composant est mis à jour, ce qui peut causer des ralentissements.

Dans notre exemple, un tableau vide de dépendances est utilisé pour indiquer que la fonction `useEffect` ne dépend d'aucune variable et ne doit être exécutée qu'une seule fois, lors du montage du composant. Cela signifie que le code contenu dans la fonction `useEffect` sera exécuté uniquement lors de la première mise à jour du composant, et non lors des mises à jour suivantes.

```
1 import React, { useState, useEffect } from 'react';
2 import { View, Text } from 'react-native';
3
4 const DataInitializer = () => {
5   const [data, setData] = useState(null);
6
7   useEffect(() => {
8     // Initialisation des données
9     const initialData = { message: 'Hello World!' };
10
11     // Mettre à jour l'état local avec les données initialisées
12     setData(initialData);
13   }, []);
14
15   return (
```

```

16   <View>
17     {data ? (
18       <Text>Données initialisées : {JSON.stringify(data)}</Text>
19     ) : (
20       <Text>Chargement des données...</Text>
21     )}
22   </View>
23 );
24 };
25
26 export default DataInitializer;

```

La fonction de rappel passée à `useEffect` initialise les données et met à jour l'état local avec les données initialisées à l'aide de la fonction `setData()`. Enfin, le rendu du composant affiche les données initialisées si elles existent, sinon un message de chargement est affiché grâce à la condition ternaire présente dans le `return`.

Méthode

C. Exercice : Quiz

[solution n°1 p.15]

Question 1

Quelle est la principale différence entre les props et le state ?

- ☐ Les props sont immuables tandis que le state est mutable
- ☐ Le state est immuable tandis que les props sont mutables
- ☐ Les props et le state sont tous les deux immuables

Question 2

Pour passer des données d'un composant parent à un composant enfant, nous utilisons des hooks.

- ☐ Vrai
- ☐ Faux

Question 3

Quel hook permet de gérer l'état dans un composant fonctionnel ?

- ☐ `useContext`
- ☐ `useEffect`
- ☐ `useState`

Question 4

Comment décomposer les props directement dans les paramètres de la fonction d'un composant fonctionnel ?

- ☐ En utilisant la décomposition (destructuring)
- ☐ En utilisant une boucle `for`
- ☐ En utilisant un opérateur de propagation (spread operator)

Question 5

Pour mettre à jour le state d'un compteur en utilisant `useState`, nous devons appeler la fonction de mise à jour du state avec la nouvelle valeur.

- ☐ Vrai
- ☐ Faux

III. Cycle de vie, vérification et communication bidirectionnelle

A. Cycle de vie et méthodes pour vérifier les props (PropTypes)

Comprendre le cycle de vie des composants fonctionnels

Les composants de classe et les composants fonctionnels ont des approches différentes pour gérer le cycle de vie. Dans les composants de classe, le cycle de vie est géré par des méthodes spécifiques, telles que `componentDidMount`, `componentDidUpdate` ou `componentWillUnmount`. Ces méthodes sont appelées automatiquement par React en fonction de l'étape de vie du composant.

En revanche, dans les composants fonctionnels, le cycle de vie est géré par les hooks, tels que `useEffect`, `useLayoutEffect`, `useReducer`, `useState`, etc. Les hooks permettent aux composants fonctionnels de gérer l'état et les effets secondaires, ainsi que de modifier leur comportement en fonction de leur état et des entrées utilisateur.

La principale différence entre les deux approches est que les composants fonctionnels sont plus simples à comprendre et à maintenir que les composants de classe. Les hooks permettent de séparer la logique d'état et de cycle de vie en deux parties distinctes, et de rendre le code plus lisible et modulaire.

Les hooks permettent également de réutiliser plus facilement la logique d'état et de cycle de vie entre les différents composants, ce qui facilite la création de composants personnalisés et de bibliothèques de composants réutilisables.

Bien que les composants de classe soient toujours utilisables, les composants fonctionnels avec les hooks offrent une alternative plus simple et plus efficace pour gérer le cycle de vie des composants en React.

Méthode	Utilisation des hooks <code>useEffect</code> et <code>useState</code>
Le hook <code>useState</code> est utilisé pour gérer l'état local d'un composant fonctionnel, tandis que le hook <code>useEffect</code> est utilisé pour effectuer des effets secondaires lors du montage, de la mise à jour et du démontage d'un composant. <code>useEffect</code> prend deux arguments : une fonction de rappel et un tableau de dépendances. La fonction de rappel est exécutée chaque fois que les dépendances changent.	
<pre>1 import React, { useState, useEffect } from 'react'; 2 import { View, Text, Button } from 'react-native'; 3 4 function Example() { 5 const [count, setCount] = useState(0); 6 7 useEffect(() => { 8 // Met à jour le titre de l'application à chaque clic sur le bouton 9 document.title = `Vous avez cliqué \${count} fois`; 10 }, [count]); // Ce tableau indique que nous voulons exécuter l'effet à chaque changement de 11 "count" 12 return (13 <View> 14 <Text>Vous avez cliqué {count} fois</Text> 15 <Button title="Cliquez ici" onPress={() => setCount(count + 1)} /> 16 </View> 17); 18 }</pre>	

Introduction à PropTypes

PropTypes est un module permettant de définir les types, les schémas d'objet et les valeurs par défaut des props que vous allez transmettre à votre composant, afin d'assurer sa cohérence et sa qualité. Il est important d'utiliser PropTypes pour réduire les erreurs potentielles dans l'application et améliorer la maintenance du code.

Méthode Vérification des props avec PropTypes

Pour vérifier les props avec PropTypes, importez le module PropTypes et définissez les types attendus pour chaque prop. Vous pouvez également utiliser des méthodes spécifiques pour valider les props en fonction de critères personnalisés.

```
1 import React from 'react';
2 import { Text } from 'react-native';
3 import PropTypes from 'prop-types';
4
5 const Greeting = ({ name, age }) => {
6   return (
7     <Text>
8     Bonjour, {name}! Vous avez {age} ans.
9     </Text>
10  );
11 };
12
13 Greeting.propTypes = {
14   name: PropTypes.string.isRequired,
15   age: PropTypes.number.isRequired,
16 };
17
18 export default Greeting;
```

Dans cet exemple, nous avons un composant fonctionnel Greeting qui accepte deux props : name et age. Nous utilisons PropTypes pour vérifier que name est une chaîne de caractères et age est un nombre. Nous avons également utilisé l'attribut isRequired pour indiquer que ces props sont obligatoires. Dans le cas où l'une serait manquante ou du mauvais type, une erreur sera générée.

B. Communication bidirectionnelle des données (de parent à enfant / d'enfant à parent)

Rappel La transmission des données parent à un composant enfant

Pour transmettre des données d'un composant parent à un composant enfant, nous utilisons des props. Les props permettent de passer des valeurs et des fonctionnalités d'un composant à un autre, créant ainsi une communication unidirectionnelle des données.

Fondamental Transmission des données d'un composant enfant à un composant parent

Pour transmettre des données d'un composant enfant à un composant parent, nous utilisons généralement des fonctions de rappel (callbacks). Le composant parent passe une fonction au composant enfant via les props, et le composant enfant appelle cette fonction pour transmettre des données ou signaler un événement au composant parent.

Méthode Utilisation des fonctions de rappel (callbacks)

Les fonctions de rappel sont des fonctions qui sont passées en tant que props à un composant enfant. Ces fonctions permettent au composant enfant d'interagir avec le composant parent en appelant la fonction de rappel. De cette façon, le composant parent peut réagir aux actions effectuées dans le composant enfant et mettre à jour son propre état en conséquence.


```

1 import React, { useState } from 'react';
2 import { View, Text, TextInput, Button } from 'react-native';
3
4 // Définition du composant parent
5 const ParentComponent = () => {
6   const [message, setMessage] = useState('');
7
8   // Définition de la fonction callback "handleMessage" qui prend en paramètre "text"
9   const handleMessage = (text) => {
10     // Mise à jour de l'état "message" avec la valeur de "text"
11     setMessage(text);
12   };
13
14   return (
15     <View>
16       <Text>Message reçu: {message}</Text>
17       <ChildComponent onSendMessage={handleMessage} />
18     </View>
19   );
20 };
21
22 // Définition du composant enfant
23 const ChildComponent = ({ onSendMessage }) => {
24   const [text, setText] = useState('');
25
26   const handlePress = () => {
27     // Appel de la fonction callback "onSendMessage" avec la valeur de l'état "text" en
28     paramètre
29     onSendMessage(text);
30   };
31
32   return (
33     <View>
34       <TextInput onChangeText={setText} value={text} />
35       <Button title="Envoyer" onPress={handlePress} />
36     </View>
37   );
38 };
39 export default ParentComponent;

```

Dans cet exemple, nous avons deux composants fonctionnels : `ParentComponent` et `ChildComponent`. Le `ParentComponent` possède un état appelé « *message* » qui est mis à jour à partir du `ChildComponent`. Pour cela, nous utilisons la fonction de rappel `handleMessage()` qui est passée au `ChildComponent` via la prop `onSendMessage`. Lorsque l'utilisateur appuie sur le bouton « *Envoyer* » dans le `ChildComponent`, la fonction de rappel est appelée avec le texte saisi et le message est mis à jour dans le `ParentComponent`. Le fonctionnement serait similaire dans le cas où le composant parent et enfant serait dans des fichiers distincts.

Méthode

Complément **Liens utiles**

Composants et props¹

En savoir plus sur les hooks²

Gestion de l'état dans une application React : Faire remonter l'état³

Les validations proposées par PropTypes⁴

C. Exercice : Quiz

[solution n°2 p.16]

Question 1

Quel hook permet de gérer les effets de bord et le cycle de vie d'un composant fonctionnel ?

- ☐ useEffect
- ☐ useState
- ☐ useContext

Question 2

Pour transmettre des données d'un composant enfant vers un composant parent, nous modifions directement l'état du composant parent depuis le composant enfant.

- ☐ Vrai
- ☐ Faux

Question 3

Quel est le principal avantage des fonctions de rappel (callbacks) dans la communication entre composants ?

- ☐ Elles permettent d'accéder directement à l'état du composant parent
- ☐ Elles facilitent la communication bidirectionnelle entre composants
- ☐ Elles rendent les composants plus performants

Question 4

Dans un composant fonctionnel, comment devons-nous utiliser le hook useState pour gérer l'état local ?

- ☐ En déclarant une variable d'état et une fonction de mise à jour
- ☐ En appelant la fonction useState avec l'état initial comme argument

Question 5

Les composants de classe utilisent des méthodes de cycle de vie, tandis que les composants fonctionnels utilisent des hooks.

- ☐ Vrai
- ☐ Faux

1 <https://fr.reactjs.org/docs/components-and-props.html>

2 <https://fr.reactjs.org/docs/hooks-intro.html>

3 <https://fr.reactjs.org/docs/lifting-state-up.html>

4 <https://fr.reactjs.org/docs/typechecking-with-proptypes.html>

IV. L'essentiel

Dans ce cours, nous avons étudié les notions clés de React Native 0.71.3 pour développer des applications mobiles robustes et maintenables. Nous avons mis l'accent sur l'importance des props et du state pour gérer les données et assurer une communication efficace entre les composants. Le cycle de vie des composants fonctionnels a également été abordé, permettant une meilleure compréhension de la dynamique d'une application React Native.

Les hooks `useState` et `useEffect` ont été introduits pour gérer l'état local et le cycle de vie des composants fonctionnels, offrant une approche plus moderne et flexible. Nous avons souligné l'importance de valider les props à l'aide de `PropTypes` pour garantir la cohérence des données et la fiabilité du code.

De plus, nous avons exploré la communication bidirectionnelle entre les composants parents et enfants, en utilisant des callbacks pour faciliter l'échange de données. Ces notions fondamentales permettent de créer des applications React Native évolutives et adaptées aux besoins des entreprises, tout en renforçant les compétences des développeurs dans ce domaine en pleine croissance.

V. Auto-évaluation

A. Exercice

Vous travaillez pour une entreprise spécialisée dans le développement d'applications de gestion de tâches pour les petites entreprises. Vous devez développer un composant fonctionnel « *TaskList* » qui affichera la liste des tâches à accomplir, et un composant fonctionnel « *TaskItem* » qui affichera les détails d'une tâche individuelle. Les composants doivent être reliés entre eux et permettre la communication bidirectionnelle des données.

Question 1

[solution n°3 p.17]

Complétez le composant « *TaskItem* » ci-dessous avec la validation des props à l'aide de `PropTypes` :

```
1 import React, { useState } from 'react';
2 import PropTypes from 'prop-types';
3
4 const TaskItem = (props) => {
5   const { task } = props;
6   const [isCompleted, setIsCompleted] = useState(task.isCompleted);
7
8   return (
9     <div>
10      <h3>{task.title}</h3>
11      <p>{task.description}</p>
12      <p>{task.dueDate}</p>
13      <input
14        type="checkbox"
15        checked={isCompleted}
16        onChange={() => setIsCompleted(!isCompleted)}
17      />
18      <label>Terminée</label>
19    </div>
20  );
21 };
22
23 // TODO : compléter les propTypes
24
25 export default TaskItem;
```

Question 2

[solution n°4 p.17]

Complétez le composant « *TaskList* » ci-dessous en utilisant le hook `useEffect` pour initialiser les données des tâches et mettre à jour le state avec les données reçues :

```
1 import React, { useState, useEffect } from 'react';
2 import TaskItem from './TaskItem';
3
4 const TaskList = () => {
5   const [tasks, setTasks] = useState([]);
6
7   const initialTasks = [
8     { id: 1, title: 'Tâche 1', description: 'Description de la tâche 1' },
9     { id: 2, title: 'Tâche 2', description: 'Description de la tâche 2' },
10  ];
11
12  // TODO : Initialiser les données des tâches avec le tableau contenu dans initialTasks
13
14  return (
15    <div>
16      <h2>Liste des tâches</h2>
17      <ul>
18        {tasks.map((task) => (
19          <li key={task.id}>
20            <TaskItem task={task} />
21          </li>
22        ))}
23      </ul>
24    </div>
25  );
26 };
27
28 export default TaskList;
```

B. Test

Exercice 1 : Quiz

[solution n°5 p.17]

Question 1

Complétez le code suivant pour valider la prop `user` :

```
1 Component.propTypes = {
2   isActive: PropTypes.__(true/false),
3 };

```

- ☐ bool
- ☐ boolean
- ☐ isBoolean

Question 2

Pour utiliser `useState` et définir l'état initial de la liste des tâches, nous devons utiliser le code suivant : `const [tasks, setTasks] = useState([])`.

- ☐ Vrai
- ☐ Faux

Question 3

Complétez le code suivant pour utiliser useEffect pour mettre à jour l'état lorsqu'une nouvelle tâche est ajoutée :

```
1 _____( ) => {  
2 setTasks([...tasks, newTask]);  
3 }, [newTask]);
```

- ☐ useEffect
- ☐ useState
- ☐ useCallback

Question 4

La principale différence entre les props et le state dans un composant React est que les props sont utilisées pour définir le style d'un composant, tandis que le state est utilisé pour définir les événements du composant.

- ☐ Vrai
- ☐ Faux

Question 5

Complétez le code suivant pour transmettre une fonction de rappel (callback) d'un composant parent à un composant enfant :

```
1 // Parent component  
2 const ParentComponent = () => {  
3   const handleClick = () => {  
4     console.log('Button clicked!');  
5   };  
6  
7   return (  
8     <ChildComponent _____={handleClick} />  
9   );  
10 };  
11  
12 // Child component  
13 const ChildComponent = (props) => {  
14   return (  
15     <button onClick={props._____}>  
16       Click me  
17     </button>  
18   );  
19 };
```

- ☐ onClick, onClick
- ☐ handleClick, handleClick
- ☐ buttonCallback, buttonCallback

Solutions des exercices

Exercice p. 6 Solution n°1**Question 1**

Quelle est la principale différence entre les props et le state ?

- ☒ Les props sont immuables tandis que le state est mutable
- ☐ Le state est immuable tandis que les props sont mutables
- ☐ Les props et le state sont tous les deux immuables
-  Les props sont immuables, ce qui signifie qu'un composant enfant ne peut pas modifier les valeurs des props reçues. Le state est mutable et peut être modifié au cours du cycle de vie d'un composant.

Question 2

Pour passer des données d'un composant parent à un composant enfant, nous utilisons des hooks.

- ☐ Vrai
- ☒ Faux
-  Les hooks sont généralement utilisés pour gérer l'état et les effets de bord dans les composants fonctionnels. Pour passer des données d'un composant parent à un composant enfant, il est préférable d'utiliser des props.

Question 3

Quel hook permet de gérer l'état dans un composant fonctionnel ?

- ☐ useContext
- ☐ useEffect
- ☒ useState
-  Le hook useState est utilisé pour déclarer et gérer l'état dans un composant fonctionnel. Il prend une valeur initiale en argument et retourne un tableau contenant deux éléments : la valeur de l'état actuel et une fonction pour mettre à jour cet état.

Question 4

Comment décomposer les props directement dans les paramètres de la fonction d'un composant fonctionnel ?

- ☒ En utilisant la décomposition (destructuring)
- ☐ En utilisant une boucle for
- ☐ En utilisant un opérateur de propagation (spread operator)
-  La décomposition (destructuring) peut être utilisée pour extraire les props directement dans les paramètres de la fonction du composant. Par exemple : `const Greeting = ({ name }) => { ... }`.

Question 5

Pour mettre à jour le state d'un compteur en utilisant useState, nous devons appeler la fonction de mise à jour du state avec la nouvelle valeur.

- ☒ Vrai
- ☐ Faux

- Q Lorsque vous utilisez `useState`, une fonction de mise à jour est fournie pour modifier l'état. Pour mettre à jour le state d'un compteur, appelez cette fonction en lui passant la nouvelle valeur calculée, par exemple en ajoutant 1 à la valeur actuelle du compteur. Le hook `useEffect` permet de gérer les effets de bord et le cycle de vie d'un composant fonctionnel et `UseContext` permet de partager simplement les props entre les composants.

Exercice p. 10 Solution n°2

Question 1

Quel hook permet de gérer les effets de bord et le cycle de vie d'un composant fonctionnel ?

- ☒ `useEffect`
- ☐ `useState`
- ☐ `useContext`

- Q Le hook `useEffect` permet de gérer les effets de bord et le cycle de vie d'un composant fonctionnel. Le `useState` sert à déclarer une variable d'état dans un composant fonctionnel. `UseContext` permet de partager simplement les props entre les composants.

Question 2

Pour transmettre des données d'un composant enfant vers un composant parent, nous modifions directement l'état du composant parent depuis le composant enfant.

- ☐ Vrai
- ☒ Faux

- Q Pour transmettre des données d'un composant enfant vers un composant parent, nous utilisons des fonctions de rappel (callbacks) passées via les props.

Question 3

Quel est le principal avantage des fonctions de rappel (callbacks) dans la communication entre composants ?

- ☐ Elles permettent d'accéder directement à l'état du composant parent
- ☒ Elles facilitent la communication bidirectionnelle entre composants
- ☐ Elles rendent les composants plus performants

- Q Les fonctions de rappel facilitent la communication bidirectionnelle entre les composants, en permettant au composant enfant de transmettre des informations au composant parent.

Question 4

Dans un composant fonctionnel, comment devons-nous utiliser le hook `useState` pour gérer l'état local ?

- ☒ En déclarant une variable d'état et une fonction de mise à jour
- ☐ En appelant la fonction `useState` avec l'état initial comme argument

- Q Pour utiliser le hook `useState` dans un composant fonctionnel, il faut déclarer une variable d'état et une fonction de mise à jour, et appeler `useState` avec l'état initial comme argument.

Question 5

Les composants de classe utilisent des méthodes de cycle de vie, tandis que les composants fonctionnels utilisent des hooks.

☒ Vrai

☐ Faux

 La principale différence entre les composants de classe et les composants fonctionnels concernant le cycle de vie est que les composants de classe utilisent des méthodes de cycle de vie, tandis que les composants fonctionnels utilisent des hooks comme useEffect.

p. 11 Solution n°3

```
1 TaskItem.propTypes = {  
2   task: PropTypes.shape({  
3     title: PropTypes.string.isRequired,  
4     description: PropTypes.string.isRequired,  
5     dueDate: PropTypes.string.isRequired,  
6     isCompleted: PropTypes.bool.isRequired,  
7   }).isRequired,  
8 };
```

p. 12 Solution n°4

```
1 useEffect(() => {  
2   setTasks(initialTasks);  
3 }, []);
```

Méthode

Exercice p. 12 Solution n°5

Question 1

Complétez le code suivant pour valider la prop user :

```
1 Component.propTypes = {  
2   isActive: PropTypes.__(true/false),  
3 };
```

☒ bool

☐ boolean

☐ isBoolean

 Pour valider une prop de type booléen, nous utilisons la méthode PropTypes.bool.

Question 2

Pour utiliser useState et définir l'état initial de la liste des tâches, nous devons utiliser le code suivant : `const [tasks, setTasks] = useState([])`.

☒ Vrai

☐ Faux

- Q Pour utiliser useState et définir l'état initial de la liste des tâches, nous utilisons effectivement cette syntaxe qui permet de déclarer l'état initial de la liste des tâches comme un tableau vide et d'associer une fonction de mise à jour appelée setTasks.

Question 3

Complétez le code suivant pour utiliser useEffect pour mettre à jour l'état lorsqu'une nouvelle tâche est ajoutée :

```
1 _____(() => {
2   setTasks([...tasks, newTask]);
3 }, [newTask]);
```

- ☒ useEffect
- ☐ useState
- ☐ useCallback

- Q Pour utiliser useEffect pour mettre à jour l'état lorsqu'une nouvelle tâche est ajoutée, nous utilisons la syntaxe useEffect(() => { ... }, [newTask]);

Question 4

La principale différence entre les props et le state dans un composant React est que les props sont utilisées pour définir le style d'un composant, tandis que le state est utilisé pour définir les événements du composant.

- ☐ Vrai
- ☒ Faux

- Q La principale différence entre les props et le state est que les props sont utilisées pour transmettre des données entre les composants, tandis que le state est utilisé pour gérer les données internes d'un composant.

Question 5

Complétez le code suivant pour transmettre une fonction de rappel (callback) d'un composant parent à un composant enfant :

```
1 // Parent component
2 const ParentComponent = () => {
3   const handleClick = () => {
4     console.log('Button clicked!');
5   };
6
7   return (
8     <ChildComponent _____={handleClick} />
9   );
10 };
11
12 // Child component
13 const ChildComponent = (props) => {
14   return (
15     <button onClick={props._____}>
16       Click me
17     </button>
18   );
19 };
```

- ☒ `onButtonClick, onButtonClick`
- ☐ `handleClick, handleClick`
- ☐ `buttonCallback, buttonCallback`
- ☐ Pour transmettre une fonction de rappel (callback) d'un composant parent à un composant enfant, nous utilisons la syntaxe `<ChildComponent onButtonClick={handleButtonClick} />` et `<button onClick={props.onButtonClick}>`.